# PRODUCT REQUIREMENTS DOCUMENT

## SIH26053 — Adaptive Variable-Resolution 2.5D LiDAR Mapping for Dynamic Environment Perception

**Version:** 1.0
**Stage:** SIH 2026 Prototype
**Organisation:** DRDO
**Product Type:** Robotics / Autonomous Perception / LiDAR Mapping Software
**Working Product Name: AVR-Map — Adaptive Variable-Resolution Semantic 2.5D Mapper**

---

# 1. Executive Summary

AVR-Map is a real-time autonomous-perception prototype that converts incoming 3D LiDAR point clouds into a **hierarchical, semantic, variable-resolution 2.5D environment map**.

The central problem is that a uniform high-resolution map wastes large amounts of memory and computation by representing all regions equally, even though nearby obstacles require much greater detail than distant, well-understood terrain. Conversely, aggressively lowering resolution can remove small but safety-critical objects such as pedestrians, poles, curbs, barriers and rocks.

The prototype therefore dynamically controls map resolution.

Instead of asking only:

> "How far away is this region?"

the system asks:

> **"How much spatial detail is required to understand this region safely right now?"**

The map will use a dyadic hierarchy:

```
5 cm → 10 cm → 20 cm → 40 cm
```

with distance establishing the baseline resolution and selected safety cues refining regions when necessary.

The prototype will combine:

- LiDAR semantic segmentation
- elevation mapping
- adaptive spatial resolution
- semantic-aware refinement

- uncertainty-aware refinement
- small-object preservation
- multi-surface/overhang representation
- terrain traversability
- dynamic-object tracking
- sparse hierarchical indexing
- real-time visualisation
- benchmark comparison against uniform-resolution mapping

The SIH research identifies the variable-resolution grid engine—not semantic segmentation alone—as the central engineering problem.

The broader research further supports a hierarchical sparse 2.5D architecture whose baseline resolution is determined by distance and later selectively refined using semantic importance, uncertainty, dynamics and terrain complexity.

---

# 2. Product Vision

Build a system capable of receiving raw LiDAR scans and continuously producing a compact map that tells an autonomous ground vehicle:

```
WHAT is around me?
WHERE is it?
HOW HIGH is it?
IS IT MOVING?
IS THE TERRAIN SAFE?
HOW CONFIDENT am I?
HOW MUCH DETAIL should I preserve here?
```

while avoiding the computational cost of maintaining maximum map resolution everywhere.

---

# 3. Problem Statement

Autonomous ground vehicles receive large point clouds from LiDAR sensors.

A single point may contain:

```
x
y
z
intensity
```

A scan may contain tens or hundreds of thousands of such points.

A complete perception system may also need:

```
semantic classification
terrain estimation
object tracking
map updates
elevation calculation
traversability calculation
visualisation
navigation
```

for every incoming frame.

Three traditional representations create a trade-off.

| Representation | Advantage | Problem |
|---|---|---|
| 2D occupancy grid | Very lightweight | Loses height information |
| Uniform 2.5D grid | Retains terrain elevation | Fine resolution everywhere wastes memory |
| Full 3D voxel map | Represents complex geometry | More memory and processing than many ground-navigation scenarios require |

SIH26053 asks for a semantically meaningful, non-uniform 2.5D representation with near-field detail, farther-field coarsening, careful boundary handling and quantitative FPS/memory/accuracy evaluation.

AVR-Map addresses this with a fourth approach:

> **Sparse hierarchical multi-surface 2.5D mapping.**

---

# 4. Product Goal

The prototype must demonstrate that an adaptive map can provide useful near-field and safety-critical detail while using significantly fewer spatial cells and less processing than a uniform fine-resolution map.

The prototype is successful when judges can visually and quantitatively see:

```
Uniform high-resolution map
            VS
Adaptive AVR-Map

More cells                 Fewer cells
More memory                Lower memory
```

```
Same detail everywhere     Detail where needed
No semantic adaptation     Safety-aware refinement
```

while important objects are still preserved.

---

# 5. Prototype Philosophy

The prototype must be ambitious enough to demonstrate innovation but simple enough to remain reliable.

Therefore the V1 product will implement:

```
DISTANCE
   +
SEMANTIC IMPORTANCE
   +
UNCERTAINTY
   +
DYNAMIC OBJECT STATE
   +
TERRAIN COMPLEXITY
          ↓
RESOLUTION DECISION
```

The following will intentionally remain V2 features:

```
trajectory-aware resolution
collision-risk prediction
vehicle-speed-aware foveation
learned traversability
camera-LiDAR fusion
full 3D adaptive islands
wavelet map compression
end-to-end learned map refinement
```

Hooks should exist for them, but they must not delay the prototype.

---

# 6. Target Users

## 6.1 Autonomous Navigation Stack

Consumes:

- terrain map
- obstacle information
- traversability
- dynamic-object tracks

and may later feed these into Nav2 or another planner.

## 6.2 Robotics / Perception Engineer

Needs to inspect:

- semantic predictions
- map cells
- resolution levels
- map accuracy
- timing
- memory
- algorithm behaviour

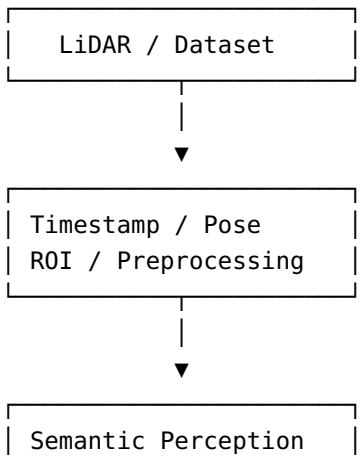
## 6.3 SIH Judge / Demonstration Viewer

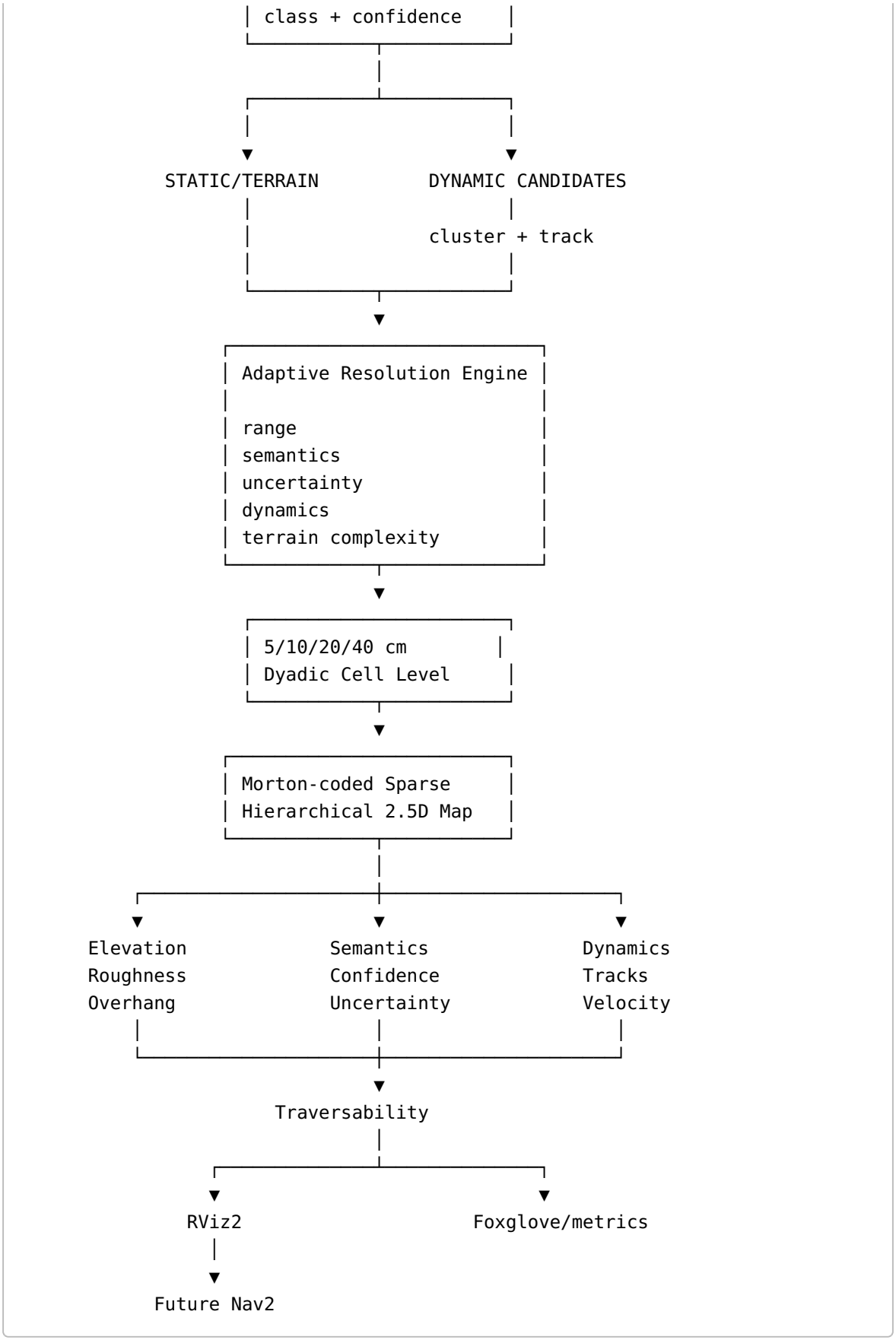
Must be able to understand the key innovation in under a few minutes.

The UI therefore needs to make adaptive resolution visually obvious.

---

# 7. Primary Product Flow




class + confidence
STATIC/TERRAIN
DYNAMIC CANDIDATES
cluster + track
Adaptive Resolution Engine
range
semantics
uncertainty
dynamics
terrain complexity
5/10/20/40 cm
Dyadic Cell Level
Morton-coded Sparse
Hierarchical 2.5D Map
Elevation
Roughness
Overhang
Semantics
Confidence
Uncertainty
Dynamics
Tracks
Velocity
Traversability
RViz2
Foxglove/metrics
Future Nav2

The research decomposes the problem similarly into pose compensation, semantics, motion, terrain geometry, vertical geometry, adaptive resolution, hierarchical indexing, traversability and whole-pipeline real-time performance.

---

# 8. Product Scope

## 8.1 Required for Prototype V1

The product SHALL support:

1. recorded LiDAR dataset playback;
2. optional real LiDAR/ROS PointCloud2 input;
3. semantic point classification;
4. semantic confidence;
5. adaptive variable-resolution cells;
6. sparse hierarchical indexing;
7. elevation estimation;
8. small-object preservation;
9. lightweight overhang representation;
10. static/dynamic separation;
11. basic object tracking;
12. geometric traversability;
13. uncertainty-aware behaviour;
14. rolling/local mapping;
15. live visualisation;
16. uniform-vs-adaptive benchmarking;
17. performance logging.

---

# 9. Explicit Non-Goals

The prototype SHALL NOT attempt to become:

- a full SLAM system;
- a production autonomous-driving stack;
- a full HD-map builder;
- a complete path-planning system;
- a camera-LiDAR fusion stack;
- a full 3D reconstruction system;
- a safety-certified perception system;
- a reinforcement-learning navigation system.

Vehicle pose/odometry should be supplied by:

- dataset ground truth,
- odometry,
- SLAM,
- or a simulated pose stream.

# 10. Input Requirements

## 10.1 LiDAR Point

Minimum:

```
x
y
z
```

Preferred:

```
x
y
z
intensity
timestamp
ring/channel
```

## 10.2 Vehicle Pose

Preferred:

```
position
orientation
timestamp
```

Used for:

- ego-motion compensation;
- accumulation;
- dynamic-object reasoning;
- stable rolling-map coordinates.

## 10.3 Input Sources

The system SHALL support at least:

### Dataset mode

Priority:

1. SemanticKITTI
2. RELLIS-3D

3. GOOSE where integrated

Secondary:

- nuScenes
- SemanticPOSS
- own rosbag

SemanticKITTI and RELLIS-3D are already identified as especially useful for urban semantic benchmarking and off-road terrain respectively.

### ROS mode

```
sensor_msgs/PointCloud2
nav_msgs/Odometry
```

---

# 11. Dual Semantic Provider Architecture

Semantic perception SHALL be replaceable.

```
SemanticProvider
       │
       ├── GroundTruthProvider
       │
       └── NeuralNetworkProvider
```

## Why this matters

Development of the mapping algorithm must not be blocked by AI model integration.

### GroundTruthProvider

Reads labels already provided by datasets.

Purpose:

- test map logic;
- benchmark resolution;
- debug cell aggregation;
- validate small-object preservation.

### NeuralNetworkProvider

Runs real segmentation.

Recommended V1 candidate:

**SalsaNext or another validated lightweight LiDAR segmentation model.**

The architecture must allow the model to be replaced later by:

- MinkUNet
- SPVCNN
- Cylinder3D
- SphereFormer
- another model

without rewriting the map engine.

---

# 12. Semantic Point Representation

After semantic inference:

```
SemanticPoint
{
    x
    y
    z

    intensity

    semantic_class
    confidence

    timestamp
}
```

Optional:

```
class_probability[]
```

The map engine consumes semantic points—not raw network-specific tensors.

---

# 13. Resolution Hierarchy

The prototype SHALL use dyadic cell sizes:

```
Level 0 = 0.05 m
Level 1 = 0.10 m
```

```
Level 2 = 0.20 m
Level 3 = 0.40 m
```

Optional future:

```
Level 4 = 0.80 m
```

Using powers of two ensures that cell boundaries remain naturally nested. The research specifically recommends snapping adaptive resolution to power-of-two multiples because parent/child boundaries then align by construction.

---

# 14. Prototype Base Resolution Policy

For the initial prototype:

```
distance <= 10 m
→ 5 cm

10–30 m
→ 10 cm

30–60 m
→ 20 cm

60–100 m
→ 40 cm
```

These values are implementation defaults, not claims of theoretical optimality.

They SHALL be configurable in:

```
config/resolution.yaml
```

Future versions may replace the fixed bands with a LiDAR-density-derived function.

---

# 15. Adaptive Refinement Logic

Distance establishes only the **starting level**.

Cells may be refined further.

Pseudo-logic:

```
level = base_level(distance)

if safety_critical_semantic:
    level = refine(level, 1 or 2)

if tracked_dynamic_object:
    level = min(level, 1)

if high_uncertainty AND sufficient_point_density:
    level = refine(level, 1)

if high_geometric_complexity:
    level = refine(level, 1)

return clamp(level)
```

---

# 16. Safety-Critical Semantic Refinement

Examples:

```
pedestrian
cyclist
vehicle
pole
traffic cone
barrier
curb
small obstacle
```

Class availability depends on the dataset/model.

Example:

```
80 m road
base = 40 cm
final = 40 cm
```

but:

```
80 m pole
base = 40 cm
semantic refinement = two levels
final = 10 cm
```

The system therefore avoids blindly removing important objects simply because they are far away.

---

# 17. Uncertainty-Aware Refinement

Low confidence SHALL NOT automatically mean:

```
create maximum-resolution cells
```

because finer cells cannot create missing sensor information.

Instead:

```
IF
confidence low
AND
point density sufficient

THEN
refine one level
```

otherwise:

```
mark region UNKNOWN
```

This distinction is crucial.

```
uncertainty + information
→ refine

uncertainty + insufficient observations
→ unknown
```

---

# 18. Dynamic-Object Refinement

Tracked moving objects should maintain at least:

```
10 cm
```

resolution wherever sufficient LiDAR returns exist.

A moving object therefore remains locally detailed even when travelling through an otherwise coarse region.

---

# 19. Terrain-Complexity Refinement

Cells should refine when geometry indicates:

```
large elevation variance
strong height discontinuity
rough surface
step
local obstacle
```

This acts as a backup when semantic segmentation is imperfect.

---

# 20. Refinement Hysteresis

Immediate repeated:

```
refine
coarsen
refine
coarsen
```

must be avoided.

Refinement may happen immediately.

Coarsening SHALL require stability for multiple observations.

Example logic:

```
importance detected
→ refine immediately

importance disappears
→ start stability counter

counter >= configurable threshold
→ coarsen
```

This avoids map flickering.

---

# 21. Spatial Data Structure

The product SHALL NOT use a naïve pointer-heavy quadtree as its primary runtime representation.

Instead use a:

> **Morton-coded tiled linear quadtree / sparse hierarchical grid**

Logical structure:

```
40 cm
┌──────────────────────────┐
│                          │
│   20 cm       20 cm      │
│                          │
│           ┌────┬────┐    │
│           │10cm│10cm│    │
│           ├────┼────┤    │
│           │10cm│10cm│    │
│           └────┴────┘    │
└──────────────────────────┘
```

but runtime storage resembles:

```
hash(
    level,
    morton(tile_x, tile_y)
)
    ↓
contiguous Tile
    ↓
cells[]
```

---

# 22. Morton / Z-Order Key

Example:

```
x bits = 1 0 1
y bits = 0 1 1

interleave

1 0 0 1 1 1
```

This creates a spatial key.

Conceptual key:

```
struct CellKey
{
    uint8_t level;
    uint64_t morton_xy;
};
```

---

# 23. Tile Design

Instead of hashing every individual cell, use tiles.

Example:

```
Tile
16 × 16 cells
```

Tile key:

```
(level, tile_x, tile_y)
```

Advantages:

- lower hash overhead;
- contiguous memory;
- improved cache locality;
- easier iteration;
- future GPU processing;
- easier map pruning.

---

# 24. Boundary Alignment Rules

The system SHALL follow these invariants:

```
one shared origin

+

power-of-two resolutions
```

```
+

integer coordinate quantisation

+

half-open cell bounds [min,max)

+

one active leaf owns each location
```

Result:

```
no intentional gaps
no intentional overlap
no duplicate observation ownership
```

Research explicitly identifies these principles as a clean solution to variable-resolution boundary consistency.

---

# 25. Point-to-Cell Assignment Algorithm

For every point:

```
1. transform point to local map frame

2. calculate range

3. calculate baseline level

4. apply semantic refinement

5. apply uncertainty refinement

6. apply dynamic/geometry refinement

7. convert x/y to integer base-grid coordinates

8. shift coordinates according to resolution level

9. generate Morton key

10. locate/create tile
```

```
11. update cell statistics
```

Pseudo-code:

```
for point in semantic_points:

    d = sqrt(x² + y²)

    level = distance_level(d)

    level = semantic_refine(level, point.class)

    level = uncertainty_refine(
        level,
        point.confidence,
        local_density
    )

    level = dynamic_refine(level)

    cell = map.lookup(level, x, y)

    cell.update(point)
```

---

# 26. Active Leaf Ownership

When an area has been refined:

```
parent
   ↓
inactive for direct observations

children
   ↓
active
```

Measurements must not simultaneously update:

```
parent
AND
child
```

as independent observations.

Parents may retain cached summaries for fast visualization/querying.

---

# 27. Raw Point Replay Cache

A major prototype feature SHALL be a short recent-point buffer.

Problem:

```
40 cm cell receives points

later:
semantic obstacle detected

cell subdivides to 10 cm

BUT

original point distribution is gone
```

This would create empty refined cells.

Therefore each active tile should retain a small recent point cache.

Example:

```
latest 2–5 scans
```

configurable.

When refinement occurs:

```
split parent
      ↓
retrieve recent raw points
      ↓
re-bin points into children
```

This preserves actual detail.

---

# 28. Multi-Surface 2.5D Cell

A cell should not contain only:

```
height = 1 value
```

because road and elevated obstacles may occupy the same XY area.

The research explicitly identifies single-height elevation maps as insufficient for multilevel geometry.

Therefore AVR-Map SHALL use a lightweight multi-surface representation.

Conceptual structure:

```
struct Cell
{
    float ground_z;
    float ground_variance;

    float z_min;
    float z_max;

    float overhang_low_z;
    float overhang_high_z;

    float roughness;

    float occupancy;

    uint8_t semantic_class;
    float semantic_confidence;

    float uncertainty;

    float traversability;

    float dynamic_probability;

    uint32_t last_seen;

    uint16_t point_count;

    uint8_t resolution_level;

    uint16_t flags;
};
```

Actual packed memory size SHALL be measured rather than assumed.

---

# 29. Ground Surface Estimation

Ground candidates should be derived from:

- semantic terrain/road classes;
- local elevation statistics;
- geometric continuity.

Within a candidate ground set:

```
ground_z
```

may be estimated using:

- robust percentile;
- median;
- probabilistic mean update.

The map SHALL maintain:

```
ground_z
ground_variance
```

not just one unqualified mean.

The research recommends keeping multiple geometric statistics because mean, median, percentiles and variance solve different failure cases.

---

# 30. Semantic Fusion

Never overwrite semantic state using only:

```
latest frame wins
```

Instead accumulate semantic evidence.

Conceptually:

```
previous class evidence
+
new class probability × confidence weight
=
updated evidence
```

The prototype may store either:

### Compact mode

```
top class
confidence
secondary hazard flag
```

or:

### Research mode

```
small probability vector
```

The first is preferred for V1 memory efficiency.

---

# 31. Small-Object Preservation

This is a major product requirement.

Consider:

```
cell contains:

95 road returns
3 pole returns
2 curb returns
```

Naïve majority:

```
ROAD
```

Critical information disappears.

Instead the cell SHALL support:

```
dominant_class = road

secondary_hazard = pole

hazard_probability = high
```

```
refinement_required = true
```

Small-object safety events must not be eliminated by majority aggregation.

---

# 32. Overhang Representation

A normal cell may contain:

```
tree branch
──────────────── 2.4m

free space

road
──────────────── 0.0m
```

The map SHALL maintain:

```
ground_z

overhang_low_z

overhang_high_z
```

Then:

```
vertical_clearance =
overhang_low_z - ground_z
```

The map can therefore identify:

```
road is traversable horizontally

BUT

vehicle may not fit vertically
```

---

# 33. Overhang Candidate Logic

An overhead surface is a candidate when:

```
drivable ground exists

AND

elevated static returns exist

AND

no evidence indicates that the obstacle is ground-connected
```

The prototype must label this as an estimate, not absolute 3D truth.

Complex environments such as:

```
parking garages
multiple bridges
interlocking vertical surfaces
```

are V2/full-3D-hybrid problems.

---

# 34. Static Map vs Dynamic Layer

The system SHALL maintain separate conceptual layers.

```
STATIC MAP

terrain
road
building
pole
persistent obstacle
elevation
```

and:

```
DYNAMIC LAYER

pedestrian
moving vehicle
temporary obstacle
tracks
velocity
short-lived occupancy
```

A pedestrian walking through a road cell must not leave permanent pedestrian occupancy.

Research explicitly recommends keeping dynamic observations out of persistent terrain fusion.

---

# 35. Dynamic Candidate Extraction

Semantic categories may identify objects *capable* of movement.

They do NOT prove movement.

Example:

```
class = car
```

does not imply:

```
moving = true
```

Candidates are clustered using:

```
DBSCAN
```

or:

```
Euclidean clustering
```

to create object hypotheses.

---

# 36. Motion Tracking

Prototype tracker:

```
semantic cluster
      ↓
3D centroid / bounding box
      ↓
Kalman Filter
      ↓
prediction
      ↓
Hungarian association
```

```
      ↓
track update
```

Each track stores:

```
track_id
class
position
velocity
confidence
age
last_seen
```

Motion decision:

```
speed > configurable threshold
→ moving
```

after ego-motion compensation.

---

# 37. Dynamic Object Map Behaviour

Dynamic object:

```
detected
   ↓
high local resolution
   ↓
temporary occupancy
   ↓
track
   ↓
prediction
```

When track disappears:

```
dynamic occupancy decays quickly
```

instead of contaminating the persistent map.

---

# 38. Traversability Engine

Traversability must answer:

> “Can THIS vehicle safely move through this cell?”

Therefore it must be parameterised by the target vehicle.

Vehicle profile:

```
vehicle_width

vehicle_height

ground_clearance

maximum_step_height

maximum_slope

safety_margin
```

---

# 39. Traversability Inputs

The V1 score should consider:

```
slope
roughness
step height
ground variance
semantic class
occupancy
vertical clearance
uncertainty
```

---

# 40. Traversability Hard Vetoes

Certain conditions should override weighted scoring.

Examples:

```
wall
→ NOT traversable
```

```
step > vehicle capability
→ NOT traversable
```

```
slope > vehicle capability
→ NOT traversable
```

```
vertical clearance < required clearance
→ NOT traversable
```

```
unknown high-risk cell
→ UNKNOWN / conservative
```

---

# 41. Traversability Score

For cells without a hard veto:

```
cost =
    ws × normalized_slope
  + wr × normalized_roughness
  + wh × normalized_step
  + wm × semantic_cost
  + wu × uncertainty

traversability =
    1 - clamp(cost, 0, 1)
```

Weights SHALL be configurable.

Output categories:

```
0.75–1.00 → high traversability

0.40–0.75 → caution

0.00–0.40 → poor

hard veto → blocked
```

```
insufficient information → unknown
```

Thresholds remain configurable and are demonstration parameters rather than universal vehicle-safety values.

---

# 42. Map Coordinate Frame

Use:

```
local odom frame
```

for stored map coordinates.

The vehicle moves within the map.

The active window follows the vehicle by allocating new tiles and deleting old ones.

Do NOT continually transform every stored cell into `base_link`.

Coordinate relationship:

```
odom
  ↓
base_link
  ↓
lidar
```

via ROS `tf2`.

---

# 43. Rolling Local Map

Map radius:

```
configurable
```

prototype target:

```
approximately 100 m
```

Old tiles outside the active window SHALL:

```
expire
or
be recycled
```

This ensures bounded memory usage.

---

# 44. Stale Cell Handling

Every cell has:

```
last_seen
```

Confidence decays over time.

Example:

```
recent observation
→ normal confidence

old observation
→ lower confidence

very stale
→ UNKNOWN/remove
```

Dynamic information decays much faster than static terrain.

---

# 45. ROS 2 System Architecture

Recommended implementation:

```
/avrm
```

package group.

Nodes:

```
lidar_input_node

preprocess_node
```

```
semantic_node

dynamic_tracker_node

adaptive_map_node

traversability_node

benchmark_node

visualization_node
```

---

# 46. ROS Topics

Suggested interfaces:

```
/points_raw

/points_filtered

/points_semantic

/odometry

/adaptive_map

/traversability_map

/dynamic_tracks

/map_metrics
```

Optional:

```
/uniform_baseline_map
```

---

# 47. Technology Stack

## Robotics

```
ROS 2
tf2
```

```
sensor_msgs
nav_msgs
RViz2
```

## Point Cloud

```
PCL
or
Open3D where useful
```

## AI

```
PyTorch
```

Inference abstraction should later support:

```
ONNX Runtime
TensorRT
```

## Core Mapping

Preferred:

```
C++17+
```

because map insertion, cell aggregation and real-time iteration are performance-sensitive.

## AI Node

Preferred:

```
Python
```

initially.

## Visualization

Required:

```
RViz2
```

Recommended demonstration layer:

```
Foxglove Studio
```

---

# 48. Recommended Repository Structure

```
AVR-Map/
│
├── README.md
│
├── docs/
│   ├── PRD.md
│   ├── architecture.md
│   ├── algorithms.md
│   └── benchmarks.md
│
├── config/
│   ├── resolution.yaml
│   ├── vehicle.yaml
│   ├── semantic.yaml
│   └── traversability.yaml
│
├── ros2_ws/
│   └── src/
│
│       ├── avr_msgs/
│       │
│       ├── avr_input/
│       │
│       ├── avr_preprocessing/
│       │
│       ├── avr_semantics/
│       │
│       ├── avr_mapping/
│       │
│       ├── avr_tracking/
│       │
│       ├── avr_traversability/
│       │
│       ├── avr_benchmark/
│       │
│       └── avr_visualization/
│
├── models/
│
├── scripts/
│
```

```
├── datasets/
│
├── rviz/
│
├── foxglove/
│
├── tests/
│
└── results/
```

---

# 49. Core Prototype Feature — Twin Map Benchmark Mode

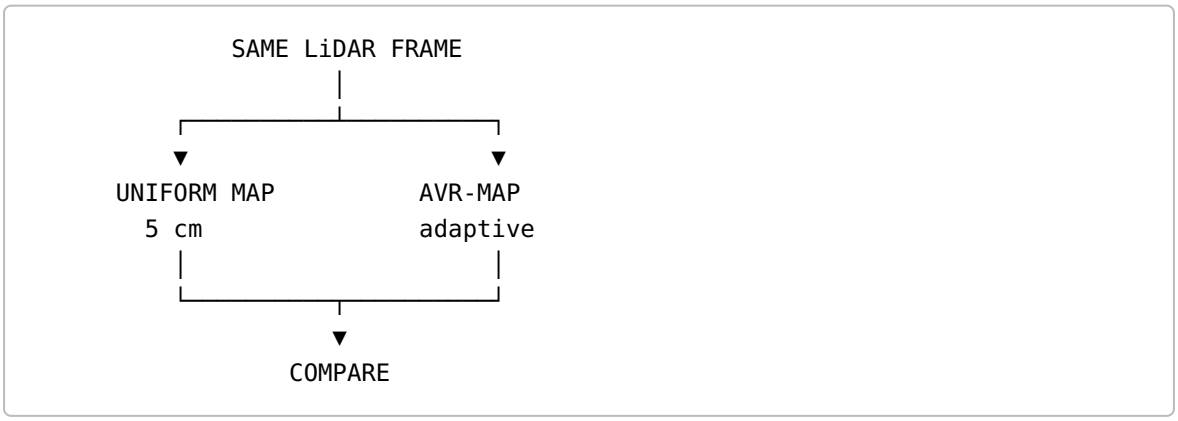
This feature is REQUIRED.

The same LiDAR stream SHALL be processed by:

```
A. Uniform fine-resolution baseline

B. Adaptive AVR-Map
```

Example:




Measured simultaneously:

```
cell count

memory

map update time

FPS
```

```
latency

small-object preservation

elevation error
```

This provides judges with direct evidence.

---

# 50. Dashboard Requirements

The demonstration dashboard should display:

```
Current FPS

Semantic inference ms

Mapping update ms

End-to-end latency

Active map cells

Map memory

Uniform baseline cells

Uniform baseline memory

Memory saved %

Current dynamic tracks

Current high-resolution cells

Current medium-resolution cells

Current coarse cells
```

---

# 51. RViz Layers

Toggleable visualisations:

```
Raw LiDAR
```

```
Semantic LiDAR

Adaptive grid

Resolution heatmap

Elevation

Traversability

Dynamic tracks

Overhangs

Uncertainty

Uniform baseline
```

---

# 52. Resolution Colour Mode

Judges should visually see the hierarchy.

Example:

```
5 cm cells
→ one colour

10 cm
→ second colour

20 cm
→ third colour

40 cm
→ fourth colour
```

The exact palette is a UI choice.

---

# 53. Semantic Colour Mode

Example categories:

```
road / terrain
vegetation
```

```
vehicle
pedestrian
building
pole
unknown
```

Dataset label mappings SHALL be configurable.

---

# 54. Cell Inspector

Clicking/selecting a cell should expose:

```
resolution

ground elevation

maximum elevation

overhang height

semantic class

confidence

uncertainty

point count

traversability

dynamic probability

last update
```

This is extremely useful for technical demonstration.

---

# 55. Required Demo Scenario 1 — Memory Reduction

Run identical scene:

```
Uniform 5cm
```

```
VS

AVR-Map
```

Show:

```
active cells
RAM estimate/actual allocation
map update latency
```

Do not present theoretical savings as measured results.

---

# 56. Required Demo Scenario 2 — Small Object Preservation

Scene contains:

```
pole
cone
pedestrian
or similar small hazard
```

far enough that ordinary distance mapping would become coarse.

Show:

```
distance only
→ coarse

semantic AVR
→ local refinement
```

Object remains represented.

---

# 57. Required Demo Scenario 3 — Uncertainty

Artificial or genuine low-confidence area.

Show:

```
low confidence + observations
→ refine

low confidence + sparse observations
→ UNKNOWN
```

This demonstrates that uncertainty is used operationally.

---

# 58. Required Demo Scenario 4 — Moving Object

Show:

```
car/person
      ↓
track ID
      ↓
velocity arrow
      ↓
temporary dynamic occupancy
```

After object leaves:

```
road becomes free again
```

---

# 59. Required Demo Scenario 5 — Overhang

Scene containing:

```
branch
boom barrier
bridge
or synthetic overhanging structure
```

Display:

```
ground height

overhang height

vertical clearance
```

# 60. Required Demo Scenario 6 — Rough Terrain

Using RELLIS-3D/GOOSE or equivalent:

show:

```
smooth terrain
→ traversable

rough terrain
→ reduced score

steep/blocked
→ blocked
```

# 61. Functional Requirements

| ID | Requirement |
|---|---|
| FR-01 | System shall ingest sequential LiDAR point clouds |
| FR-02 | System shall support dataset playback |
| FR-03 | System shall accept or derive semantic labels |
| FR-04 | System shall maintain semantic confidence |
| FR-05 | System shall assign cells to multiple spatial resolutions |
| FR-06 | Cell resolutions shall use a nested hierarchy |
| FR-07 | System shall support local refinement |
| FR-08 | System shall support coarsening |
| FR-09 | System shall preserve safety-critical semantic evidence |
| FR-10 | System shall maintain elevation statistics |
| FR-11 | System shall represent lightweight overhang information |
| FR-12 | System shall estimate traversability |
| FR-13 | System shall maintain a dynamic-object layer |
| FR-14 | System shall track moving objects temporally |
| FR-15 | System shall maintain uncertainty |
| FR-16 | System shall maintain stale-data timestamps |

| ID | Requirement |
|---|---|
| FR-17 | System shall support rolling-map operation |
| FR-18 | System shall visualise all major layers |
| FR-19 | System shall compare adaptive vs uniform mapping |
| FR-20 | System shall record benchmark metrics |

---

# 62. Non-Functional Requirements

## Performance

Target:

```
>= 10 Hz end-to-end
```

on the chosen benchmark hardware for a 10 Hz input stream.

Stretch:

```
15–20 Hz
```

No claim is valid until measured.

## Latency

Prototype target:

```
p95 end-to-end latency <= one sensor frame period
```

for the chosen input frequency.

## Reliability

The map must not crash when:

- no points arrive;
- semantic output is missing;
- a tile is empty;
- odometry temporarily pauses;
- class confidence is low.

### Determinism

Given:

```
same input
same configuration
same model output
```

the mapping engine should produce reproducible cell assignments.

---

# 63. Prototype Acceptance Criteria

The prototype is considered complete when:

#### Mapping

- variable-resolution map is operational;
- 5/10/20/40 cm levels are visible;
- boundaries contain no observable gaps/duplicate ownership.

#### Semantics

- semantic labels can enter from either GT or neural inference.

#### Small objects

- at least one safety-critical object demonstrates local refinement.

#### Dynamics

- at least one moving object receives an ID and estimated velocity.

#### Terrain

- elevation and traversability layers render correctly.

#### Overhang

- at least one controlled overhang is represented through clearance information.

#### Benchmark

Adaptive and uniform maps report:

```
cell count
memory
```

```
latency
FPS
```

from the same input.

### Performance

The complete pipeline keeps up with the selected demonstration playback rate.

---

# 64. Benchmark Metrics

## Mapping

```
active cells

allocated tiles

memory bytes

map update latency

elevation RMSE where GT exists
```

## Semantic

```
mIoU
per-class IoU

or

ground-truth semantic preservation rate
```

depending on experiment.

## Small Objects

Define:

```
Small Object Preservation Rate =
objects retained after mapping
/
objects present before aggregation
```

Test across distance bands.

### Traversability

Where labels exist:

```
accuracy
precision
recall
F1
```

Safety-critical false-positive traversability must be highlighted separately.

### Tracking

Where ground truth exists:

```
position error
velocity error
track continuity
ID switches
```

### System

```
FPS

mean latency

p50

p95

CPU

GPU

RAM

VRAM
```

---

# 65. Primary Benchmark

The headline comparison SHALL be:

```
UNIFORM 5 CM
vs
ADAPTIVE 5/10/20/40 CM
```

using exactly:

```
same map radius
same input
same semantic output
same temporal window
same machine
```

Only then are:

```
memory savings
latency savings
cell reduction
```

meaningful.

---

# 66. Secondary Ablation Experiments

Compare:

```
distance only

vs

distance + semantics
```

then:

```
distance + semantics

vs

distance + semantics + uncertainty
```

then:

```
with / without dynamic refinement
```

This tells us which adaptive mechanisms actually help.

---

# 67. Failure Modes

The system must explicitly consider:

```
semantic misclassification

LiDAR sparsity at long range

rain/fog/dust

vegetation

reflective surfaces

rapid ego motion

localisation error

small-object loss

overhang ambiguity

stale cells

tracker ID switch

map thrashing

GPU overload

missing odometry
```

---

# 68. Critical Failure — Semantic Refinement Cascade

Example:

```
pedestrian
     ↓
misclassified as road
     ↓
```

```
road considered safe
     ↓
coarsened
     ↓
pedestrian spatial information disappears
```

Mitigation:

```
geometry-based hazard detection

uncertainty refinement

secondary hazard evidence

minimum feature-preservation logic
```

This failure must be tested rather than hidden.

---

# 69. Critical Failure — Fake Detail

Problem:

```
far region
contains almost no points

system refines to 5cm
```

Result:

```
more cells
NOT more knowledge
```

Mitigation:

```
point-density gate
```

Refinement requires sufficient evidence.

Otherwise:

```
UNKNOWN
```

---

# 70. Critical Failure — Resolution Thrashing

Mitigation:

```
immediate refine

delayed coarsen

hysteresis
```

---

# 71. Critical Failure — Stale Dynamic Occupancy

Mitigation:

```
short TTL

fast decay

separate dynamic layer
```

---

# 72. Development Sequence

## Phase A — Deterministic Mapping Core

Use ground-truth dataset semantics.

Implement:

```
input
adaptive hierarchy
Morton indexing
aggregation
RViz
uniform comparison
```

No neural inference dependency.

## Phase B — Multi-Surface Terrain

Implement:

```
ground statistics
roughness
small-object preservation
overhang
traversability
```

## Phase C — Semantic Inference

Connect:

```
SalsaNext or selected model
```

through the `SemanticProvider`.

## Phase D — Dynamics

Implement:

```
clustering
Kalman
Hungarian association
dynamic layer
```

## Phase E — Adaptive Intelligence

Add:

```
uncertainty refinement
geometric refinement
hysteresis
point replay
```

## Phase F — Benchmark & Demo

Implement:

```
uniform baseline
metrics
recorded experiments
Foxglove/RViz layouts
SIH demo scenes
```

---

# 73. Configuration-Driven Design

No important threshold should be hard-coded.

Example:

```
map:
  radius_m: 100

resolution:
  levels:
    - 0.05
    - 0.10
    - 0.20
    - 0.40

  bands:
    - max_distance: 10
      level: 0
    - max_distance: 30
      level: 1
    - max_distance: 60
      level: 2
    - max_distance: 100
      level: 3

uncertainty:
  refine_threshold: configurable

dynamic:
  max_resolution_level: 1

coarsening:
  stability_frames: configurable
```

---

# 74. Product Differentiators

AVR-Map should not be described simply as:

> “A LiDAR semantic mapping application.”

Its differentiation is the integration of:

### Adaptive spatial information allocation

Detail is allocated where information matters.

### Semantic-aware resolution

Object meaning influences map resolution.

### Uncertainty-aware behaviour

Unknown areas are handled explicitly.

### Small-object protection

Minority safety signals cannot simply disappear through coarse aggregation.

### Multi-surface 2.5D

Vertical clearance can be represented without full 3D mapping everywhere.

### Dynamic/static separation

Temporary actors do not corrupt persistent terrain.

### Measurable efficiency

The product proves its advantage against an actual baseline.

---

# 75. Judge-Facing One-Line Product Description

**AVR-Map converts LiDAR data into a real-time semantic 2.5D map that automatically spends high spatial resolution on nearby, uncertain, dynamic or safety-critical regions while compressing well-understood distant terrain, preserving navigation-critical information with lower memory and processing cost.**

---

# 76. Judge-Facing Example

Imagine:

```
80 m away:
flat empty road
```

AVR-Map:

```
40 cm
```

Now:

```
80 m away:
pole detected
```

AVR-Map:

```
40 cm
→ semantic importance
→ 20 cm
→ 10 cm
```

Now:

```
40 m ahead:
unknown rough region
on driveable surface
```

AVR-Map:

```
20 cm
→ uncertainty/geometry
→ 10 cm
```

Now:

```
70 m:
almost no LiDAR returns
```

AVR-Map does NOT pretend to understand it better.

```
insufficient data
→ UNKNOWN
```

This single example communicates the core philosophy of the product.

---

# 77. Future V2 Features

After the SIH prototype is stable:

```
trajectory-aware refinement

collision-risk-aware refinement
```

```
speed-aware foveation

hybrid 2.5D + sparse 3D islands

learned traversability

camera/LiDAR fusion

TensorRT deployment

GPU map updates

advanced moving-object segmentation

adaptive per-cell information compression

global map persistence
```

---

# 78. Potential Research Extension

A later research version may formalise an adaptive importance function:

```
I =
    αS
  + βU
  + γD
  + δG
  + εP
  + ζR
```

where:

```
S = semantic importance
U = uncertainty
D = dynamic probability
G = geometric complexity
P = path relevance
R = collision risk
```

Then:

```
resolution level =
f(distance, sensor density, I)
```

This is deliberately outside the initial prototype.

The V1 should first prove that a smaller subset of these signals provides measurable benefit.

---

# 79. Definition of Done

AVR-Map V1 is complete when the team can run one command or launch file and show:

```
LiDAR entering system

↓

semantic point cloud

↓

adaptive variable-resolution map

↓

terrain + elevation

↓

small-object refinement

↓

moving-object tracking

↓

traversability

↓

overhang clearance

↓

uniform vs adaptive benchmark

↓

live memory / cell / latency / FPS numbers
```

with no manual intervention during the primary demonstration.

---

# 80. Final Prototype Decision

The prototype we should build is therefore:

# Adaptive Semantic Multi-Surface 2.5D LiDAR Mapping System

with:

```
SemanticKITTI / RELLIS / ROS input

+

replaceable semantic provider

+

5 / 10 / 20 / 40 cm dyadic hierarchy

+

distance baseline

+

semantic refinement

+

uncertainty refinement

+

terrain-complexity refinement

+

dynamic-object refinement

+

Morton-coded tiled sparse hierarchy

+

multi-surface elevation cells
```

```
+

small-object preservation

+

dynamic/static separation

+

Kalman + Hungarian tracking

+

vehicle-aware traversability

+

RViz2 / Foxglove visualisation

+

uniform-vs-adaptive benchmark mode
```

This gives the team a prototype that is both **buildable and technically meaningful**, while leaving the much harder path/risk/speed-aware adaptive system as a logical research extension rather than risking an unstable SIH demonstration.